Clasificación de los cinco fragmentos.

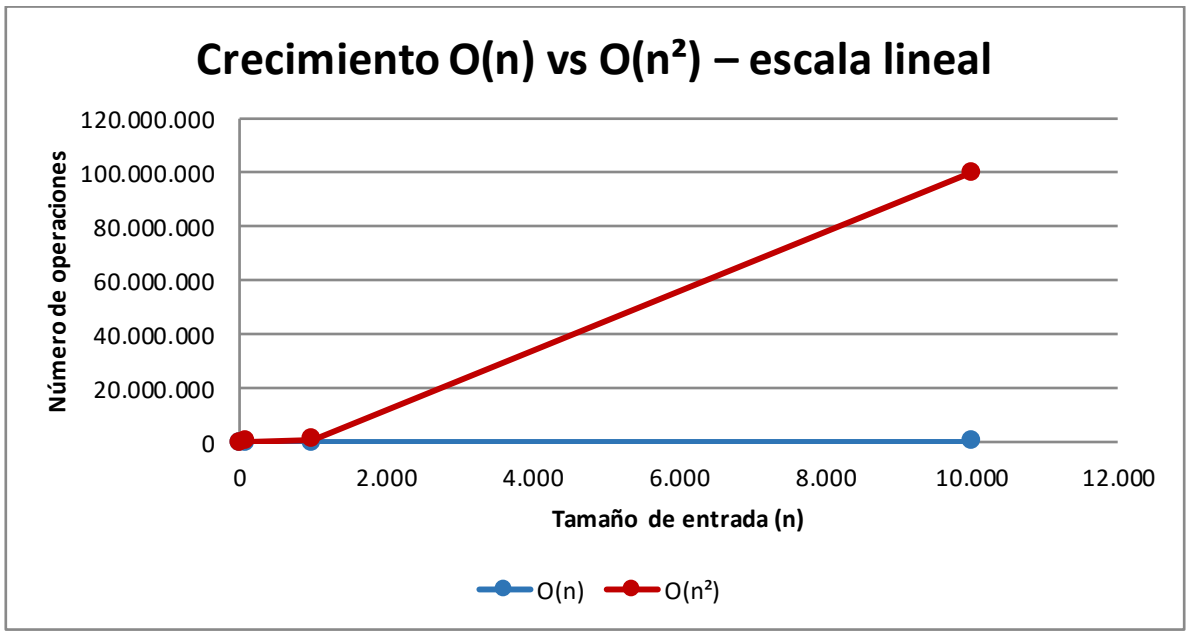
Fragmento	Operaciones aproximadas	Big O	Justificación
Acceder a vector[5]	1	O(1)	Consulta una sola posición, sin importar el tamaño del arreglo.
Recorrer un arreglo	n	O(n)	Un ciclo visita cada elemento una vez, así que el trabajo crece proporcional a n.
Dos ciclos anidados	$n \times n = n^2$	$O(n^2)$	Por cada iteración del ciclo externo, el interno ejecuta n iteraciones.
Dividir n entre 2 repetidamente	$\log_2 n$	O(log n)	En cada paso el problema se reduce a la mitad, por lo que los pasos crecen muy lentamente.
Ciclo n + reducción logarítmica	$n \times \log_2 n$	O(n log n)	El ciclo externo se ejecuta n veces y, en cada una, el interno hace unas log n divisiones (como en el Ejercicio 5).

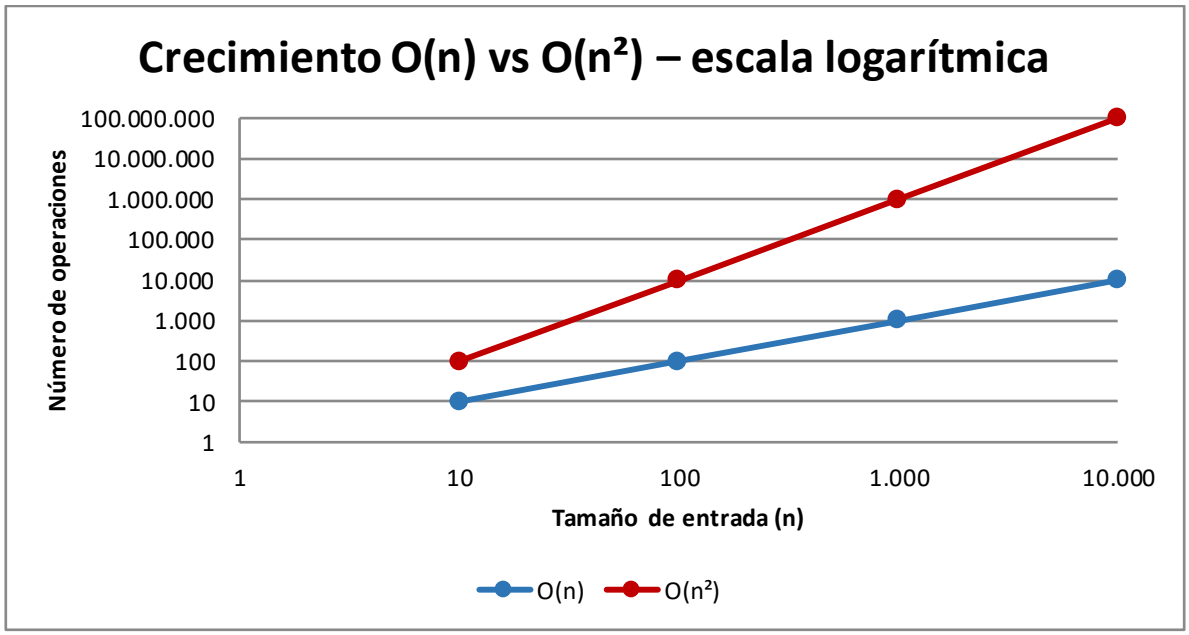
Grafica comparativa

n	O(n)	$O(n^2)$	Razón $O(n^2)$ / O(n)
10	10	100	10
100	100	10.000	100
1.000	1.000	1.000.000	1.000
10.000	10.000	100.000.000	10.000

Observaciones

1. Cuando n crece 10 veces (de 10 a 100), O(n) crece 10 veces, pero $O(n^2)$ crece 100 veces
2. Con n = 10.000, O(n) es solo 10.000 operaciones, mientras que $O(n^2)$ llega a 100.000.000.
3. La razón $O(n^2)/O(n)$ es igual a n: la brecha entre ambos se agranda a medida que n crece.
4. En el gráfico lineal, O(n) parece casi plana pegada al eje X y $O(n^2)$ se dispara: ese es el crecimiento explosivo.
5. En el gráfico logarítmico ambas curvas se ven como rectas; $O(n^2)$ tiene el doble de pendiente que O(n)
6. Conclusión: un algoritmo $O(n^2)$ no escala; con entradas grandes conviene buscar uno O(n) o O(n log n).





1. ¿Diferencia entre tiempo de ejecución y complejidad Big O? El tiempo de ejecución es lo que se mide en el computador (en mi caso, nanosegundos) y cambia según el procesador, la memoria y otros procesos. Big O describe cómo crece el número de operaciones cuando aumenta n, sin depender de la máquina.

2. ¿Por qué O(1) es eficiente? Porque el trabajo no cambia con el tamaño de la entrada. Acceder a datos [0] cuesta lo mismo con 10 elementos que con 1.000.000.

3. ¿Qué pasa con O(n) al duplicar el tamaño? El tiempo se duplica aproximadamente. En mi experimento, de n = 5.000 a n = 10.000 el tiempo pasó de 162.000 a 316.900 ns, es decir, unas 1,96 veces, casi el doble.

4. ¿Qué pasa con $O(n^2)$ al duplicar n? Teóricamente el trabajo se cuadruplica, porque $(2n)^2 = 4n^2$. En mi experimento, de 5.000 a 10.000 el tiempo pasó de 6.618.100 a 14.656.200 ns, unas 2,2 veces. No llegó a 4 porque los tiempos medidos tienen ruido

(calentamiento de la JVM, optimizaciones, otros procesos), pero se ve que crece mucho más rápido que el lineal.

5. ¿Por qué los ciclos anidados se relacionan con $O(n^2)$? Porque por cada una de las n iteraciones del ciclo externo, el interno ejecuta otras n. Eso da $n \times n = n^2$ operaciones. En el Algoritmo B, con n = 10.000 se hacen 100.000.000 de incrementos.

6. ¿Qué característica permite reconocer un algoritmo O(log n)? Que en cada paso el problema se reduce a la mitad (o a una fracción), como en n = n / 2. Por eso el número de pasos crece muy lentamente.

7. ¿Cuál algoritmo presentó mayor crecimiento? El Algoritmo B, $O(n^2)$. Entre n = 100 y n = 10.000, O(n) pasó de 6.700 a 316.900 ns (unas 47 veces) y $O(n^2)$ pasó de 362.700 a 14.656.200 ns (unas 40 veces en tiempo medido, aunque en operaciones teóricas crece 10.000 veces). En todos los tamaños el cuadrático fue más lento: con n = 10.000 tardó unas 46 veces más que el lineal.

8. ¿Los resultados fueron exactamente iguales entre ejecuciones? ¿Por qué? No. Los tiempos varían por el procesador, la memoria disponible, el sistema operativo, los procesos en ejecución y las optimizaciones de Java (JIT). En mis datos se nota: $O(n^2)$ con n = 1.000 tardó 8.953.900 ns y con n = 5.000 tardó 6.618.100 ns, aunque teóricamente 5.000 debería tardar bastante más. Probablemente fue por el calentamiento de la JVM en las primeras ejecuciones. Por eso no se debe confundir el tiempo medido con la complejidad Big O.

9. Si tuviera que procesar diez millones de registros, ¿qué complejidad preferiría? O(n) o O(n log n). Con n = 10.000.000, $O(n^2)$

requeriría unos 10^{14} operaciones. Con la velocidad que medí en mi computador (unos 14,6 ms para 10^8 operaciones), eso serían del orden de varias horas, mientras que un O(n) se resolvería en una fracción de segundo.

10. ¿Por qué un ingeniero debe analizar la eficiencia antes de elegir un algoritmo? Porque un algoritmo puede funcionar bien con pocos datos y volverse inutilizable con muchos, como se vio con $O(n^2)$. Elegir mal afecta el tiempo de respuesta, el consumo de recursos y los costos. Funcionar no es suficiente: hay que saber cómo se comportará cuando crezca la información.

Conclusiones

Antes de hacer esta actividad pensaba que si un programa daba el resultado correcto ya estaba bien. Ahora veo que eso es solo una parte. Al correr el experimento me di cuenta de que con n = 100 los dos terminaron en menos de un milisegundo, pero con n = 10.000 el cuadrático tardó 14.656.200 ns y el lineal solo 316.900 ns, unas 46 veces más. Si con diez mil datos ya se nota así, con millones sería un problema serio.

Algo que me llamó la atención fue que los tiempos no siguieron exactamente lo que dice la teoría. Por ejemplo, el cuadrático con n = 1.000 tardó más que con n = 5.000. Entendí que lo que se mide depende del computador, de la memoria y hasta de cómo la JVM optimiza el código, y que por eso Big O no habla de segundos sino de cómo crece el trabajo cuando n aumenta.

Me llevo la idea de que antes de elegir un algoritmo hay que pensar en cuántos datos va a manejar. Un código que hoy funciona puede volverse inútil cuando la aplicación crezca. Prefiero analizar antes que descubrir el problema cuando ya es tarde.